

4.3. ПРОГРАММИРОВАНИЕ ВЛОЖЕННЫХ УСЛОВИЙ

ВЫ НАУЧИТЕСЬ

Использовать вложенные условия на языке программирования Python (пайтон).

КЛЮЧЕВЫЕ СЛОВА

Вложенные условия	Кірістірілген шарттар	Nested conditions
Операторы сравнения	Салыстыру операторлары	Comparison operators
Каскадные ветвления	Каскадты тармақталу	Cascading branches



Вам известно, что с помощью разветвляющихся алгоритмов можно реализовать как полный, так и неполный алгоритм ветвления. Если возникает необходимость сделать выбор больше, чем из двух путей, то в таком случае реализуется **множественное ветвление**. Приведем пример из жизни.

Допустим, вам дали задание, которое рекомендует просмотр видеоконтента в Интернете. Просмотр видеоконтента рекомендуется проводить в зависимости от возраста пользователя по годам, например, от 2 – мультфильмы, от 5 – развивающие игры, от 8 – робототехника, от 11 – программирование в Python, от 14 – программирование игр, от 17 – финансовая грамотность.



Попробуйте составить словесный алгоритм решения задачи с помощью конструкции ветвления.

При решении задачи вы будете применять несколько раз условный оператор ветвления **if**. Он будет появляться внутри другого оператора **if**. В таких случаях операторы считаются **вложенными**.

Вложенное ветвление – это ветвление, находящееся в одной из ветвей другого ветвления.



Во вложенном ветвлении после одной развилки в ходе исполнения программы появляется другая развилка. При этом вложенные блоки имеют больший размер отступа (например, 8 пробелов). Из-за неэффективности выполнения вложенных операторов рекомендуют использовать один-два уровня вложения (рис. 4.22).

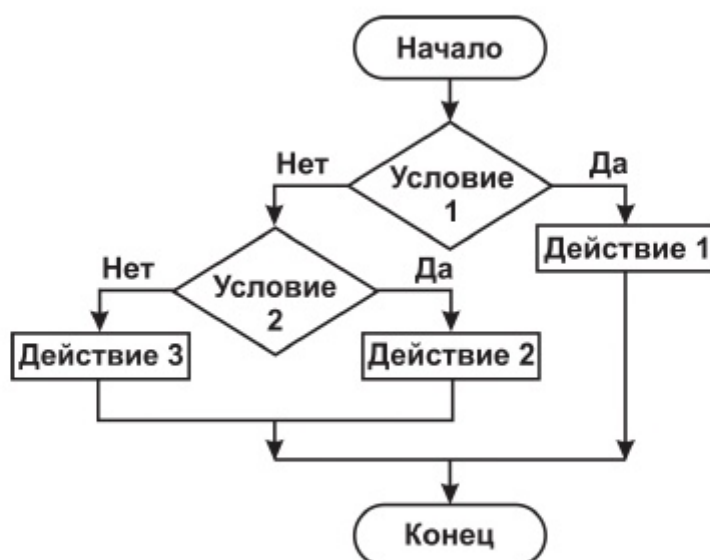


Рис. 4.22. Блок-схема вложенного ветвления

В конструкции вложенных операторов каждому оператору **if** соответствует только один оператор **else**. Совокупность этих операторов **else-if** означает, что если не выполняется предыдущее условие, то проверяется данное. Если ни одно из условий неверно, то выполняется тело оператора **else**.

Структура вложенных операторов

```

if <условие 1>:
    <инструкции 1>
else
    if <условие 2>:
        <инструкции 2>
    else
        <инструкции 3>
  
```



Определите, в какой из четвертей координатной плоскости находится точка с координатами (x, y) .

Шаг за шагом



1. Введите числа **x** и **y**.
 2. Разберите конструкцию вложенного ветвления, поскольку необходимо проверить более одного условия:

1) Если точка располагается в **первой** координатной четверти, то ее координаты удовлетворяют условию **$x > 0, y > 0$** .

2) Если точка располагается в **четвертой** координатной четверти, то ее координаты удовлетворяют условию **$x > 0, y < 0$** .

3) Если точка располагается во **второй** координатной четверти, то ее координаты удовлетворяют условию **$x < 0, y > 0$** .

4) Если точка располагается в **третьей** координатной четверти, то ее координаты удовлетворяют условию **$x < 0, y < 0$** .

3. Первое и второе действия можно объединить в один **if**, так как у них одинаковое **условие $x > 0$** . В зависимости от значения **y** точка будет лежать в **первой** или в **четвертой** координатной четверти. Таким образом получается **вложенное ветвление**.

4. Если условие **$x > 0$** является ложным, то выполняются операторы после **else**, где вновь выполняется проверка условия **$y < 0$** или **$y > 0$** , и от этого условия зависит, во **второй** или **третьей** четверти будет находиться точка.

5. Напишите код программы (рис. 4.23) с вложенным условием.

```

File Edit Format Run Options Window Help
x = int(input())
y = int(input())
if x > 0:
    if y > 0:
        print("Первая четверть")
    else:
        print("Четвертая четверть")
else:
    if y > 0:
        print("Вторая четверть")
    else:
        print("Третья четверть")
Ln: 12 Col: 32
1
1
Первая четверть
>>>
===== RESTART: F:/1
2
-2
Четвертая четверть
>>>
===== RESTART: F:/1
-3
3
Вторая четверть
>>>
===== RESTART: F:/1
-5
-3
Третья четверть

```

Рис. 4.23. Код программы вложенного ветвления



По заданному количеству глаз и ножек нужно отличить собаку, паука, морского гребешка и жука. У морского гребешка бывает более сотни глаз, а у паука их восемь. Также у паука восемь ног, а у морского гребешка их нет совсем. У собаки четыре ноги, а у жука – шесть ног, но глаз у них по два.

Внутри блока команд могут находиться другие условные операторы. Рассмотрим пример.

На рис. 4.24 представлен код программы. Вложенных условных операторов в программе несколько. К какому из них относится **else**, можно увидеть по отступу. Отступ у **else** должен быть такой же, как у **if**, к которому он относится.

```

File Edit Format Run Options Window Help
eyes = int(input())
legs = int(input())
if eyes >= 8:
    if legs == 8:
        print("spider")
    else:
        print("scallop")
else:
    if legs == 6:
        print("bug")
    else:
        print("dog")

----- RESTART: -----
8
8
spider
>>>
----- RESTART: -----
100
0
scallop
>>>

----- RESTART: -----
2
6
bug
>>>
----- RESTART: -----
2
4
dog
>>>

```

Рис. 4.24. Код программы «Глаза и ножки»

В некоторых ситуациях необходимо осуществить выбор больше чем из двух вариантов. В язык Python встроена возможность множественного ветвления на одном уровне вложенности, которое реализуется с помощью специальной конструкции **if... elif... else**.

Оператор **elif** образован от двух первых букв оператора **else**, к которым присоединено слово **if**. Это можно перевести как «иначе если». После него записывается **условие**. Конструкций **elif** может быть несколько, условия в них проверяются последовательно. Как только выполняется условие, запускается соответствующий этому условию блок команд и дальнейшая проверка не выполняется.

Блок **else** является необязательным, как и в обычном **if**. В отличие от **else**, в заголовке **elif** обязательно должно быть логическое выражение, как в заголовке **if**.

Такая конструкция введена в язык Python для улучшения читаемости. При применении записи **if-else** увеличивается количество отступов, что приводит к ухудшению читаемости. Например, программу, определяющую для точки четверть координатной плоскости, можно оптимизировать, используя «каскадную» последовательность операций **if... elif... else**.

Каскадные ветвления используются для разделения кода более чем на две ветви.



Структура каскадного ветвления

```
if <условие 1>:
    <инструкции 1>
elif <условие 2>:
    <инструкции 3>
...
else:
    <инструкции 2>
```

Алгоритм выполнения каскадного ветвления показан на рис. 4.25.

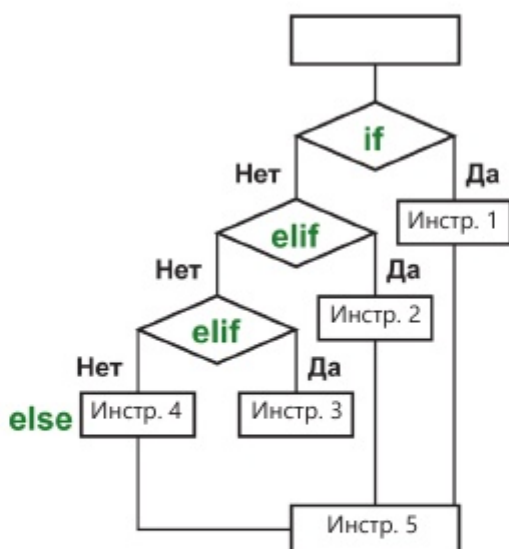


Рис. 4.25. Блок-схема конструкции **if-elif-...-elif-else**

- 1) Сначала проверяется условие **1** в блоке **if**.
- 2) Если **условие 1** выполняется, то будет выполнена **инструкция 1**, и программа вернется в основную ветку **инструкции 5**.
- 3) Если **условие 1** не выполняется, то проверяется **условие 2** в блоке **elif**.
- 4) Если **условие 2** выполняется, то будет выполнена **инструкция 2**, и программа вернется в основную ветку **инструкции 5**.
- 5) Если **условие 2** не выполняется, то будет выполнено **условие 3**.
- 6) Если **условие 3** выполняется, то выполняется **инструкция 3**.
- 7) Если **условие 3** не выполняется, то будет выполнена **инструкция 4** из блока **else**.

Приведем пример.



При вводе числа **1** или **2** необходимо вывести словом название числа или что это другое число (рис. 4.26).

```
File Edit Format Run Options Window ===== RESTART
number = int(input())
1
if number == 1:
    print('Один')
    Один
    >>>
elif number == 2:
    print('Два')
    ===== RESTART
    2
    Два
    >>>
else:
    print('Другое число')
    ===== RESTART
    100
    Другое число
    >>>
```

Рис. 4.26. Вывод чисел с сообщением



Знание. Понимание



1. Произнесите ключевые слова на трех языках.
2. Какие условия называются вложенными?
3. Перечислите структуру вложенных операторов.
4. Каким образом выполняются вложенные операторы?



5. Что такое каскадное ветвление?
6. Перечислите структуру каскадного ветвления.
7. Поясните принцип работы каскадного ветвления.
8. Почему удобнее использовать каскадное ветвление?

Применение. Анализ



9. Составьте программу для вычисления значений функции, используя вложенные ветвления:



$$y = \begin{cases} x^2, & \text{если } x > 1 \\ 2x + 1, & \text{если } x < -2, \\ -3x + 5, & \text{в остальных случаях} \end{cases}$$

10. Напишите программу, которая запрашивает на ввод число. Если оно положительное, то на экран выводится цифра **1**. Если число отрицательное, выводится **-1**. Если введенное число **0**, то на экран выводится **0**. Используйте вложенное ветвление.

Синтез. Оценивание



11. Даны три целых числа. Определите, сколько среди них совпадающих. Программа должна вывести одно из чисел: **3** (если все совпадают), **2** (если два совпадают) или **0** (если все числа различны). Используйте вложенное и каскадное ветвления. Сравните и оцените полученный результат.
12. Определите возможность существования треугольника по трем сторонам. Составьте программу, применяя вложенное или каскадное ветвление.



Дополнительные задания по программированию



1. Дано целое число **n**. Выведите следующее за ним четное число.

10-5997

2. Составьте программу для вычисления функции, заданной графиком (рис. 4.27), применяя вложенное или каскадное ветвление.

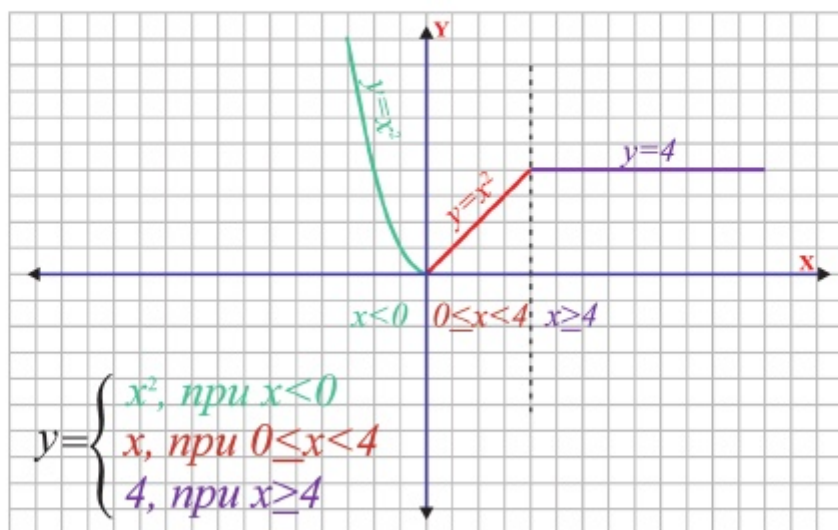


Рис. 4.27. График функции

3. Напишите программу, которая получает на вход три целых числа и выводит в три строки сначала максимальное, потом минимальное, после чего – оставшееся число. На ввод могут подаваться и повторяющиеся числа. Проанализируйте, какой способ решения данной задачи является наиболее оптимальным.
4. Дано четырехзначное число. Определите, является ли его десятичная запись симметричной. Если число симметричное, то выведите **1**, иначе выведите любое другое целое число. Число может иметь меньше четырех знаков. Тогда нужно считать, что его десятичная запись дополняется слева нулями.

4.4. ПРОГРАММИРОВАНИЕ СОСТАВНЫХ УСЛОВИЙ

ВЫ НАУЧИТЕСЬ

Использовать составные условия на языке программирования Python (пайтон).

КЛЮЧЕВЫЕ СЛОВА

Логическое высказывание	Логикалық тұжырым	Logical statement
Логическая операция	Логикалық амал	Logical operation
Составное условие	Күрделі шарт	Compound condition



Как вам известно, условие – это логическое выражение, состоящее из логических операций. Условие может быть **простым** или **составным**. Простое условие состоит из двух чисел, двух переменных или двух логических выражений, которые сравниваются между собой с использованием операций сравнения ($=$, $>$, $<$, $>=$, $<=$, $<>$). Например: $1 < 2$ (Истина), $5 < 0$ (Ложь), $x >= 5$, $x * y == x + y$.

Логическое высказывание – это утверждение, истинность которого можно определить однозначно.



Часто при решении некоторых задач возникает необходимость проверять выполнение двух и более условий.

Как вы думаете, какое условие будет называться составным?



Составное логическое выражение можно записывать так:

if $a > 0$ and $a < b$:

Это означает, что вложенный оператор выполняется, если **a** будет одновременно **больше нуля** и **меньше b**. В Python, в отличие от других языков программирования, существует также сокращенная запись составного логического выражения в виде **if $0 < a < b$:**

Рассмотрим конкретный пример.



В зависимости от возраста человек проходит разные стадии обучения, которые должны быть выведены в виде сообщений. Выделим условно группы, которые разделены на четыре диапазона:

- 1) от 3 до 6 лет – детский сад;
- 2) от 6 до 10 – начальная школа;
- 3) от 10 до 15 – основная школа;
- 4) от 15 и больше – старшая школа.

Как реализовать задачу с конструкцией ветвления с составным условием, показано на рис. 4.28.

```
old = int(input('Введите Ваш возраст: '))
print('Ваша ступень образования:', end=' ')

if 3 <= old < 6:
    print("Детский сад")
elif 6 <= old < 10:
    print("Начальная школа")
elif 10 <= old < 15:
    print("Основная школа")
elif 15 <= old:
    print("Старшая школа")
```

Введите Ваш возраст: 3
Ваша ступень образования: "Детский сад"
>>>

Рис. 4.28. Конструкция ветвления с составным условием

Из кода программы вы видите, что условие состоит из нескольких простых условий, которые объединены логическими операторами. Такие условия называют **составными**.

Для записи составного условия применяются логические операции **and** (и), **or** (или), **not** (не), которые возвращают **Истина (True)** или **Ложь (False)** операнду:

Операнды: истина (T)=1, ложь (F)=0		и (логическое умножение)	или (логическое сложение)	не (логическое отрицание)
x	y	x and y	x or y	not x
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Чтобы лучше понимать, как работают составные условия со сложными логическими выражениями, разберите приведенные ниже примеры с их решениями (табл. 4.2).

Таблица 4.2. Примеры с их решениями

Пример	Решение
$2 > 5$ and $2 > 3$	$2 > 5 = \text{False}$, $2 > 3 = \text{False}$, $\text{False and False} = \text{False}$
$1 < 5$ and $1 > 0$	$1 < 5 = \text{True}$, $1 > 0 = \text{True}$, $\text{True and True} = \text{True}$
$3 \geq 2$ or $3 < 1$	$3 \geq 2 = \text{True}$, $3 < 1 = \text{False}$, $\text{True or False} = \text{True}$
$1 > 2$ or $1 < 0$	$1 > 2 = \text{False}$, $1 < 0 = \text{False}$, $\text{False or False} = \text{False}$
not $5 > 6$	$5 > 6 = \text{False}$, not $\text{False} = \text{True}$
not $6 > 5$	$6 > 5 = \text{True}$, not $\text{True} = \text{False}$

Приведем еще один пример, в котором нужно определить значение логического выражения: **not($x > y$) and not ($x < 0$) or ($y \geq 10$)**, если $x = -1$, $y = 10$.

На порядок действий влияют также скобки. Желательно не составлять очень сложные логические выражения и всегда разбивать их на части. Для решения примера отметим порядок выполнения операций (см. табл. 4.3).

Таблица 4.3. Решение логического выражения

Порядок выполнения	4	1	6	5	2	7	3
Логическое выражение	not	($x > y$)	and	not	($x < 0$)	or	($y \geq 10$)
Выполняем в скобках		$(-1 > 10)$			$(-1 < 0)$		$(10 \geq 10)$
		F			T		T
Выполняем not	not (F)			not (T)			
	T			F			
Выполняем and			T and F				
			F				
Выполняем or						F or T	
Ответ						True	

Приведем код программы с использованием логических операций по определению точки в одной из четвертей координатной плоскости, рассмотренной в предыдущей теме (рис. 4.29).

```

File Edit Format Run Options Window Help
print("Введите координаты точки")
x = int(input())
y = int(input())
if x > 0 and y > 0:
    print("Первая четверть")
elif x > 0 and y < 0:
    print("Четвертая четверть")
elif y > 0:
    print("Вторая четверть")
else:
    print("Третья четверть")

>>>
===== RESTART: F:/7класс
Введите координаты точки
4
5
Первая четверть
>>>
===== RESTART: F:/7класс
Введите координаты точки
-4
3
Вторая четверть
>>>
===== RESTART: F:/7класс
Введите координаты точки
-2
-3
Третья четверть
>>>
===== RESTART: F:/7класс
Введите координаты точки
5
-6
Четвертая четверть
>>>

```

Рис. 4.29. Код программы с использованием логических операций



Определите, принадлежит ли введенное число заданному интервалу $-50 < x \leq 0$ или $50 \leq x < 100$.



Шаг за шагом

1. Ввести число x .
2. Если выполняется условие $-50 < x \leq 0$ или $50 \leq x < 100$, то число принадлежит заданному промежутку.
3. Записать условие на языке программирования Python, используя логические операции $x > -50$ and $x \leq 0$ or $x \geq 50$ and $x < 100$.
Данное условие можно записать в сокращенной форме: $-50 < x \leq 0$ or $50 \leq x < 100$.
4. Вывести на экран ответ **True**, если условие истинно, и **False** в противном случае.